

# EXECUTIVE SUMMARY

Target: http://shared-endpoint.com/api/v1/users

Scan ID: TORTURE-P7-2-66508c65

Scan Date: 2026-03-04 18:16:46

Scan Duration: N/A

Total Requests Sent: 27

Avg Latency: N/A

Peak Concurrency: N/A

Findings: 3 vulnerabilities detected

Severity Breakdown: Critical: 1 | High: 1 | Medium: 1 | Low: 0

AI Inference Calls: 0

LLM Avg Latency: N/A

Circuit Breaker Activations: 0

- Overall Risk Level: High due to three identified vulnerabilities across all categories
- Most Critical Category: CRITICAL (1 vulnerability) poses highest security threat
- Immediate Actions: Prioritize remediation of the single CRITICAL issue immediately
  - Long-Term Recommendations: Implement comprehensive monitoring and regular audits for ongoing protection

## EXECUTIVE STRATEGIC IMPACT

```
{
  "strategic_attack_path_analysis": [
    "An attacker could exploit the SSRF vulnerability to pivot from internal network services, potentially gaining access to sensitive systems or data.",
    "By leveraging a CSRF attack on an authenticated user session, they might be able to perform unauthorized actions within that context, escalating privileges and maintaining persistence.",
    "The SQL injection flaw in another component of the system could provide direct database access, enabling further exploitation such as exfiltration of critical information or installation of persistent backdoors."
  ]
}
```

# DETAILED FINDINGS

## FILTER: SSRF

### Finding #1: Server-Side Request Forgery

CRITICAL

CWE: CWE-918  
CVSS Score: 9.6 (CRITICAL)

THREAT SCORE: 96/100

#### Description:

- The application exposes a publicly accessible API endpoint that allows external requests to be forwarded.
- An attacker can craft malicious HTTP requests targeting the shared endpoint, potentially accessing internal resources or servers.
- SSRF enables unauthorized data exfiltration and server compromise by manipulating request URLs.

#### Impact:

- Unauthorized access to sensitive internal systems and data
- Potential privilege escalation through compromised endpoints
- Data breaches via leaked information from targeted services

#### Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'SSRF' was intercepted to protect system integrity.

# FORENSIC ANALYSIS

Method: GET | Param: param\_0  
URL: http://shared-endpoint.com/api/v1/users  
Analysis: The application failed to validate and sanitize user input before sending it to a remote server.

Table 1: Payload Decomposition

| Component    | Value          | Technical Function                            |
|--------------|----------------|-----------------------------------------------|
| param_0      | test_payload_0 | Direct reference to target resource.          |
| Access Check | Missing        | Application fails to verify authorization.    |
| Result       | 200 OK         | Server returns data for unauthorized request. |

Payload Specifications:

Vector Category: Server-Side Request Forgery  
Raw Payload: test\_payload\_0  
Encoded: 746573745f7061796c6f61645f30  
Encoding Type: None

Reproduction Command:

```
curl -X GET 'http://shared-endpoint.com/api/v1/users'
```

HTTP Traffic Snapshot:

Request:

GET http://shared-endpoint.com/api/v1/users HTTP/1.1  
Host: target  
{}

Response:

HTTP/1.1 200 OK  
Content-Type: application/json  
  
{"status": "vulnerable", "data": "revealed"}

Remediation:

- Implement strict input validation and sanitization for all external requests.
- Restrict API endpoint URLs to only trusted domains or IP addresses.
- Employ a Web Application Firewall (WAF) with rules specifically targeting SSRF attempts.

Recommended Code Fix:

```
// Example of mitigating SSRF in Java Spring Boot application
@WebApplicationParameter("requestUri")
private String requestUri;
public void setRequestUri(String uri) {
    // Validate and sanitize the input before processing
    this.requestUri = uri;
}
```

Finding #2: Cross-Site Request Forgery

HIGH

CWE: CWE-352  
CVSS Score: 7.5 (HIGH)

THREAT SCORE: 75/100

Description:

- Vulnerability 'Cross-Site Request Forgery' detected in target endpoint.

Impact:

- Unauthorized access to system resources confirmed.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'CSRF' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: POST | Param: param\_1  
URL: http://shared-endpoint.com/api/v1/users  
Analysis: The application failed to validate and sanitize user input, allowing malicious requests.

Table 1: Payload Decomposition

| Component    | Value          | Technical Function                            |
|--------------|----------------|-----------------------------------------------|
| param_1      | test_payload_1 | Direct reference to target resource.          |
| Access Check | Missing        | Application fails to verify authorization.    |
| Result       | 200 OK         | Server returns data for unauthorized request. |

Payload Specifications:

```
Vector Category: Cross-Site Request Forgery
Raw Payload:      test_payload_1
Encoded:          746573745f7061796c6f61645f31
Encoding Type:    None
```

**Reproduction Command:**

```
curl -X POST 'http://shared-endpoint.com/api/v1/users'
```

**HTTP Traffic Snapshot:****Request:**

```
POST http://shared-endpoint.com/api/v1/users HTTP/1.1
Host: target
{}
```

**Response:**

```
HTTP/1.1 200 OK
Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}
```

**Remediation:**

- Implement strict input validation.

**Recommended Code Fix:**

```
```python
from flask import request, session, redirect, url_for, abort
from werkzeug.exceptions import NotFound

@app.route('/secure', methods=['GET', 'POST'])
def secure():
    if request.method == 'POST':
        csrf_token = request.form.get('csrf_token')
        if not request.is_csrf_safe or not csrf_token:
            abort(403)

        # Process form data securely
        ...

    return render_template('secure.html')
```
```

---

**FILTER: INJECTION & FUZZING**

Finding #3: SQL Injection

CRITICAL

CWE: CWE-89  
CVSS Score: 10.0 (CRITICAL)

THREAT SCORE: 100/100

Description:

- Vulnerability 'SQL Injection' detected in target endpoint.

Impact:

- Unauthorized access to system resources confirmed.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'SQL\_INJECTION' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: PUT | Param: param\_2  
URL: http://shared-endpoint.com/api/v1/users  
Analysis: The application failed to properly sanitize user input, allowing malicious SQL code to be injected into the query.

Table 1: Payload Decomposition

| Component    | Value          | Technical Function                            |
|--------------|----------------|-----------------------------------------------|
| param_2      | test_payload_2 | Direct reference to target resource.          |
| Access Check | Missing        | Application fails to verify authorization.    |
| Result       | 200 OK         | Server returns data for unauthorized request. |

Payload Specifications:

```
Vector Category: SQL Injection
Raw Payload:      test_payload_2
Encoded:          746573745f7061796c6f61645f32
Encoding Type:    None
```

**Reproduction Command:**

```
curl -X PUT 'http://shared-endpoint.com/api/v1/users'
```

**HTTP Traffic Snapshot:****Request:**

```
PUT http://shared-endpoint.com/api/v1/users HTTP/1.1
Host: target
{}
```

**Response:**

```
HTTP/1.1 200 OK
Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}
```

**Remediation:**

- Implement strict input validation.

**Recommended Code Fix:**

```
```python
from flask import request, g
import sqlite3

def get_db():
    if 'db' not in g:
        g.db = sqlite3.connect('app.db', detect_types=sqlite3.PARSE_DECLTYPES)
    return g.db

@app.teardown_appcontext
def teardown_db(exception):
    db = g.pop('db', None)
    if db is not None:
        db.close()

@app.route('/users/<int:user_id>')
def get_user(user_id):
    conn = get_db()
    cur = conn.cursor()
```

```
# Safe parameterized query to prevent SQL injection
cur.execute("SELECT * FROM users WHERE id = ?", (user_id,))
user = cur.fetchone()
return render_template('user.html', user=user)
'''
```

# SCAN TIMELINE

[Orchestrator] TARGET\_ACQUIRED - 2026-03-04 18:16:46

[Sigma] JOB\_ASSIGNED - 2026-03-04 18:16:46

[Kappa] LOG - 2026-03-04 18:16:46

[Beta] LOG - 2026-03-04 18:16:47

[Kappa] LOG - 2026-03-04 18:16:47

[Beta] LOG - 2026-03-04 18:16:47

[Beta] LOG - 2026-03-04 18:16:47

[Alpha] LOG - 2026-03-04 18:16:47

[Gamma] LOG - 2026-03-04 18:16:47

[Beta] LOG - 2026-03-04 18:16:47

[Kappa] LOG - 2026-03-04 18:16:47

[Alpha] LOG - 2026-03-04 18:16:47

[Gamma] LOG - 2026-03-04 18:16:47

[Gamma] LOG - 2026-03-04 18:16:48

[Kappa] LOG - 2026-03-04 18:16:48

[Kappa] LOG - 2026-03-04 18:16:48

[Kappa] LOG - 2026-03-04 18:16:48

[Beta] LOG - 2026-03-04 18:16:48

[Alpha] LOG - 2026-03-04 18:16:48

[Kappa] LOG - 2026-03-04 18:16:48

[Kappa] LOG - 2026-03-04 18:16:48

[Kappa] LOG - 2026-03-04 18:16:48

[Gamma] VULN\_CONFIRMED - 2026-03-04 18:16:48

[Gamma] VULN\_CONFIRMED - 2026-03-04 18:16:48

[Gamma] VULN\_CONFIRMED - 2026-03-04 18:16:49

[Gamma] VULN\_CONFIRMED - 2026-03-04 18:16:49

[Kappa] GI5\_LOG - 2026-03-04 18:16:56